

Chapter 3

Persistence and Databases

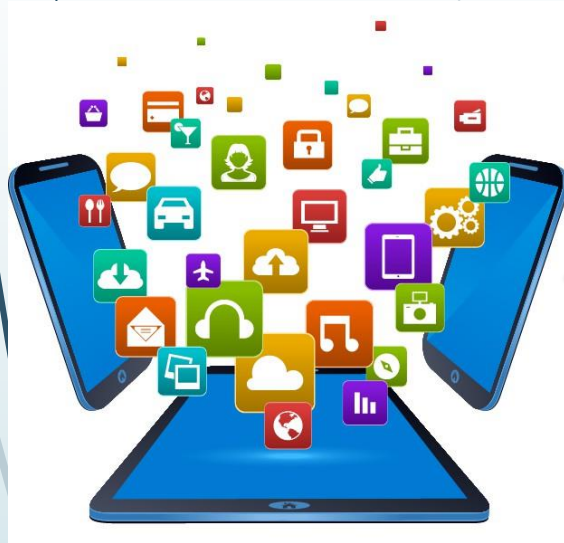
Outline

- Introduction
- Overview of the database connectivity
- Connection, Cursor, Row Objects
- Create, Read, Update and Delete (CRUD) operations
- Query Results and Metadata

Introduction

3

- ▶ A **database** is an organized collection of data.
- ▶ A **database management system (DBMS)** provides mechanisms for storing, organizing, retrieving and modifying data for many users.
- ▶ Popular **relational database management systems (RDBMSs)**
 - ▶ Microsoft SQL Server®
 - ▶ Oracle®
 - ▶ Sybase®
 - ▶ IBM DB2®
 - ▶ Informix®
 - ▶ PostgreSQL
 - ▶ MySQL™



Wake



Saved in



Connect
For java applications JDBC

Access

Use



Mostly used DBMS

- ▶ We have numberof databases so how java connect to each of this databases?
- ▶ Each DBMS has its own drivers (Library)



Introduction (cont.)

- ▶ Java programs interact with databases using the **Java Database Connectivity (JDBC™) API**.
- ▶ A **JDBC driver** enables Java applications to connect to a database in a particular DBMS and allows you to manipulate that database using the JDBC API.

JDBC

- ▶ JDBC is a **Standard API** that enables Java applications to interact with databases
- ▶ Acts as a bridge between your Java application(frontend) and the database(backend), allowing you to send and retrieve data between the two systems (Java developed system and database system)

Relational Databases

- ▶ A **relational database** is a type of database that organizes data into rows and columns, which collectively form a table where the data points are related to each other.
- ▶ A **relational database** is a logical representation of data that allows the data to be accessed without consideration of its physical structure.
- ▶ A relational database stores data in **tables**.
- ▶ **Tables** are composed of rows, each describing a single entity
- ▶ **Rows** are composed of columns in which values are stored.
- ▶ **Primary key**—a column (or group of columns) with a value that is unique for each row.

	Number	Name	Department	Salary	Location
Row {	23603	Jones	413	1100	New Jersey
	24568	Kerwin	413	2000	New Jersey
	34589	Larson	642	1800	Los Angeles
	35761	Myers	611	1400	Orlando
	47132	Neumann	413	9000	New Jersey
	78321	Stephens	611	8500	Orlando
	Primary key		Column		

Fig. 24.1 | Employee table sample data.

Department	Location
413	New Jersey
611	Orlando
642	Los Angeles

Fig. 24.2 | Distinct Department and Location data from the Employees table.

Relational Database Overview:

10

- ▶ **Entity-relationship (ER) diagram.**
 - ▶ Shows the *database tables* and the *relationships* among them.
 - ▶ The names in *italic* are primary keys.
- ▶ *A table's primary key uniquely identifies each row in the table.*
- ▶ Every row must have a primary-key value, and that value must be unique in the table.
 - ▶ This is known as the **Rule of Entity Integrity**.

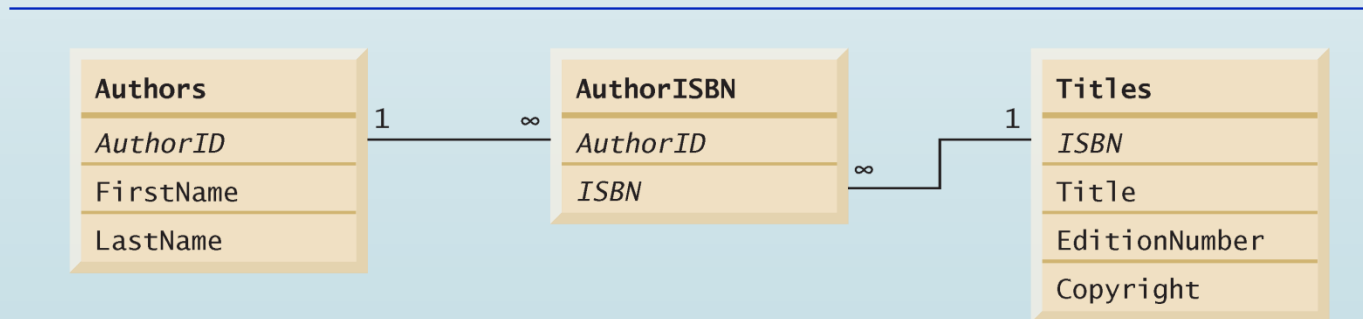


Fig. 24.9 | Table relationships in the **books** database.

SQL

- ▶ The next several subsections discuss the SQL queries and statements using the SQL keywords.
- ▶ Structured Query Language (SQL) is the standard **programming language** for interacting with relational database management systems
- ▶ allowing database administrator to **add, update, or delete** rows of data easily.

1. **SELECT**: Retrieve data from a database.

```
SELECT column1, column2 FROM table_name;
```

2. **INSERT**: Add new data to a database.

```
INSERT INTO table_name (column1, column2) VALUES (value1, value2);
```

3. **UPDATE**: Modify existing data in a database.

```
UPDATE table_name SET column1 = value1, column2 = value2 WHERE condition;
```

4. **DELETE**: Remove data from a database.

```
DELETE FROM table_name WHERE condition;
```

Basic SELECT Query

12

- ▶ A SQL query “selects” rows and columns from one or more tables in a database.
- ▶ The basic form of a **SELECT** query is
 - ▶ **SELECT** * **FROM** *tableName*
- ▶ in which the **asterisk (*) wildcard character** indicates that all columns from the *tableName* table should be retrieved.
- ▶ To retrieve all the data in the **Authors** table, use
 - ▶ **SELECT** * **FROM** Authors
- ▶ To retrieve only specific columns, replace the asterisk (*) with a comma-separated list of the column names, e.g.,
 - ▶ **SELECT** AuthorID, LastName **FROM** Authors

AuthorID	LastName	AuthorID	LastName
1	Deitel	4	Quirk
2	Deitel	5	Morgano
3	Deitel		

Fig. 24.11 | Sample AuthorID and LastName data from the Authors table.

WHERE Clause

14

- ▶ In most cases, only rows that satisfy **selection criteria** are selected.
- ▶ SQL uses the optional **WHERE clause** in a query to specify the selection criteria for the query.
- ▶ The basic form of a query with selection criteria is
 - ▶ **SELECT** *columnName1, columnName2, ...* **FROM** *tableName*
WHERE
criteria
- ▶ To select the **Title**, **EditionNumber** and **Copyright** columns from table **Titles** for which the **Copyright** date is greater than 2013, use the query
 - ▶ **SELECT** **Title**, **EditionNumber**, **Copyright**
FROM **Titles**
WHERE **Copyright** > '2013'
- ▶ Strings in SQL are delimited by single (') rather than double (") quotes.
- ▶ The **WHERE** clause criteria can contain the operators <, >, <=, >=, =, <> and **LIKE**.

Title	EditionNumber	Copyright
Java How to Program	10	2015
Java How to Program, Late Objects Version	10	2015
Visual Basic 2012 How to Program	6	2014
Visual C# 2012 How to Program	5	2014
C++ How to Program	9	2014
Android How to Program	2	2015
Android for Programmers: An App-Driven Approach, Volume 1	2	2014

Fig. 24.12 | Sampling of titles with copyrights after 2005 from table `Titles`.

WHERE Clause (cont.)

- ▶ Operator **LIKE** is used for **pattern matching** with wildcard characters **percent** (%) and **underscore** (_).
- ▶ A pattern that contains a percent character (%) searches for strings that have zero or more characters at the percent character's position in the pattern.
- ▶ For example, the next query locates the rows of all the authors whose last name starts

with the letter D:

```
▶ SELECT AuthorID, FirstName, LastName  
   FROM Authors  
   WHERE LastName LIKE 'D%'
```


AuthorID	FirstName	LastName
1	Paul	Deitel
2	Harvey	Deitel
3	Abbey	Deitel

Fig. 24.13 | Authors whose last name starts with D from the Authors table.

WHERE Clause (cont.)

- ▶ An underscore (_) in the `LIKE` pattern string indicates a single wildcard character at that position in the pattern.
- ▶ The following query locates the rows of all the authors whose last names start with any character (specified by _), followed by the letter O, followed by any number of additional characters (specified by %):
 - ▶ `SELECT AuthorID, FirstName,
 LastName FROM Authors
 WHERE LastName LIKE '_o%'`

AuthorID	FirstName	LastName
5	Michael	Morgano

Fig. 24.14 | The only author from the **Authors** table whose last name contains o as the second letter.

ORDER BY Clause

20

- ▶ The rows in the result of a query can be sorted into ascending or descending order by using the optional **ORDER BY clause**.
- ▶ The basic form of a query with an **ORDER BY** clause is
 - ▶ **SELECT** *columnName1*, *columnName2*, ... **FROM** *tableName* **ORDER BY** *column*
ASC
 - ▶ **SELECT** *columnName1*, *columnName2*, ... **FROM** *tableName* **ORDER BY** *column*
DESC
- ▶ **ASC** specifies ascending order (lowest to highest)
- ▶ **DESC** specifies descending order (highest to lowest)
- ▶ *column* specifies the column on which the sort is based.

ORDER BY Clause (cont.)

21

- ▶ To obtain the list of authors in ascending order by last name (), use the query
 - ▶ `SELECT AuthorID, FirstName, LastName
FROM Authors
ORDER BY LastName ASC`
- ▶ To obtain the same list of authors in descending order by last name (), use the query
 - ▶ `SELECT AuthorID, FirstName, LastName
FROM Authors
ORDER BY LastName DESC`
- ▶ Multiple columns can be used for sorting with an ORDER BY clause of the form
 - ▶ `ORDER BY column1 sortOrder, column2
sortOrder, ...`
 - ▶ *sortOrder* is either ASC or DESC.
- ▶ Sort all the rows in ascending order by last name, then by first name.
 - ▶ `SELECT AuthorID, FirstName, LastName
FROM Authors
ORDER BY LastName, FirstName`

AuthorID	FirstName	LastName
1	Paul	Deitel
2	Harvey	Deitel
3	Abbey	Deitel
5	Michael	Morgano
4	Dan	Quirk

Fig. 24.15 | Sample data from table **Authors** in ascending order by **LastName**.

AuthorID	FirstName	LastName
4	Dan	Quirk
5	Michael	Morgano
1	Paul	Deitel
2	Harvey	Deitel
3	Abbey	Deitel

Fig. 24.16 | Sample data from table **Authors** in descending order by **LastName**.

AuthorID	FirstName	LastName
3	Abbey	Deitel
2	Harvey	Deitel
1	Paul	Deitel
5	Michael	Morgano
4	Dan	Quirk

Fig. 24.17 | Sample data from **Authors** in ascending order by **LastName** and **FirstName**.

ORDER BY Clause (cont.)

- ▶ The **WHERE** and **ORDER BY** clauses can be combined in one query, as in
 - ▶ **SELECT** ISBN, Title, EditionNumber,
Copyright **FROM** Titles
WHERE Title **LIKE** '%How to
Program' **ORDER BY** Title **ASC**
- ▶ which returns the ISBN, Title, EditionNumber and Copyright of each book in the **Titles** table that has a **Title** ending with "How to Program" and sorts them in ascending order by **Title**.

ISBN	Title	EditionNumber	Copyright
0133764036	Android How to Program	2	2015
013299044X	C How to Program	7	2013
0133378713	C++ How to Program	9	2014
0132151006	Internet & World Wide Web How to Program	5	2012
0133807800	Java How to Program	10	2015
0133406954	Visual Basic 2012 How to Program	6	2014
0133379337	Visual C# 2012 How to Program	5	2014
0136151574	Visual C++ 2008 How to Program	2	2008

Fig. 24.18 | Sampling of books from table `Titles` whose titles end with `How to Program` in ascending order by `Title`.

Merging Data from Multiple Tables:

INNER JOIN

- ▶ Database designers often split related data into separate tables to ensure that a database does not store data redundantly.
- ▶ Often, it is necessary to merge data from multiple tables into a single result.
 - ▶ Referred to as joining the tables
- ▶ An **INNER JOIN** merges rows from two tables by matching values in columns that are common to the tables.
 - ▶

```
SELECT columnName1, columnName2, ...  
FROM table1  
INNER JOIN table2  
ON table1.columnName = table2.columnName
```
- ▶ The **ON clause** specifies the columns from each table that are compared to determine which rows are merged—these fields almost always correspond to the foreign-key fields in the tables being joined.

Merging Data from Multiple Tables:

INNER JOIN (cont.)

- ▶ The following query produces a list of authors accompanied by the ISBNs for books written by each author:

- ▶ `SELECT FirstName, LastName, ISBN FROM Authors`

```
INNER JOIN AuthorISBN
  ON Authors.AuthorID = AuthorISBN.AuthorID
ORDER BY LastName, FirstName
```

- ▶ The syntax *tableName.columnName* in the ON clause, called a **qualified name**, specifies the columns from each table that should be compared to join the tables.

INSERT Statement

- ▶ The **INSERT** statement inserts a row into a table.
 - ▶ **INSERT INTO** *tableName* (*columnName1*, *columnName2*, ..., *columnNameN*)
VALUES (*value1*, *value2*, ..., *valueN*)
- where *tableName* is the table in which to insert the row.
- ▶ *tableName* is followed by a comma-separated list of column names in parentheses
 - ▶ not required if the **INSERT** operation specifies a value for every column of the table in the correct order
 - ▶ The list of column names is followed by the SQL keyword **VALUES** and a comma-separated list of values in parentheses.
 - ▶ The values specified here must match the columns specified after the table name in both order and type.

INSERT Statement (cont.)

- ▶ The **INSERT** statement
 - ▶ **INSERT INTO** Authors (FirstName, LastName)
VALUES ('Sue', 'Red')
- ▶ indicates that values are provided for the **FirstName** and **LastName** columns. The corresponding values are 'Sue' and 'Red'.
- ▶ We do not specify an **AuthorID** in this example because **AuthorID** is an autoincremented column in the **Authors** table.
 - ▶ Not every database management system supports autoincremented columns.

UPDATE Statement

- ▶ An **UPDATE** statement modifies data in a table.
 - ▶ **UPDATE** *tableName*
 SET *columnName1 = value1, columnName2 = value2, ...,*
 columnNameN = valueN
 WHERE *criteria*
- ▶ where *tableName* is the table to update.
 - ▶ *tableName* is followed by keyword **SET** and a comma-separated list of *columnName = value* pairs.
 - ▶ Optional **WHERE** clause provides criteria that determine which rows to update.
- ▶ The **UPDATE** statement-
 - ▶ **UPDATE** Authors
 SET LastName = 'Black'
 WHERE LastName = 'Red' **AND** FirstName = 'Sue'
- ▶ indicates that **LastName** will be assigned the value **Jones** for the row where **LastName** is **Red** and **FirstName** is **Sue**.

AuthorID	FirstName	LastName
1	Paul	Deitel
2	Harvey	Deitel
3	Abbey	Deitel
4	Dan	Quirk
5	Michael	Morgano
6	Sue	Black

Fig. 24.21 | Sample data from table **Authors** after an **UPDATE** operation.

DELETE Statement

- ▶ A SQL **DELETE** statement removes rows from a table.
 - ▶ **DELETE FROM** *tableName* **WHERE** *criteria*
- ▶ where *tableName* is the table from which to delete.
 - ▶ Optional **WHERE** clause specifies the criteria used to determine which rows to delete.
 - ▶ If this clause is omitted, all the table's rows are deleted.
- ▶ The **DELETE** statement
 - ▶ **DELETE FROM** Authors
WHERE LastName = 'Black' **AND** FirstName = 'Sue'
- ▶ deletes the row for Sue Jones in the Authors table.

AuthorID	FirstName	LastName
1	Paul	Deitel
2	Harvey	Deitel
3	Abbey	Deitel
4	Dan	Quirk
5	Michael	Morgano

Fig. 24.22 | Sample data from table **Authors** after a DELETE operation.

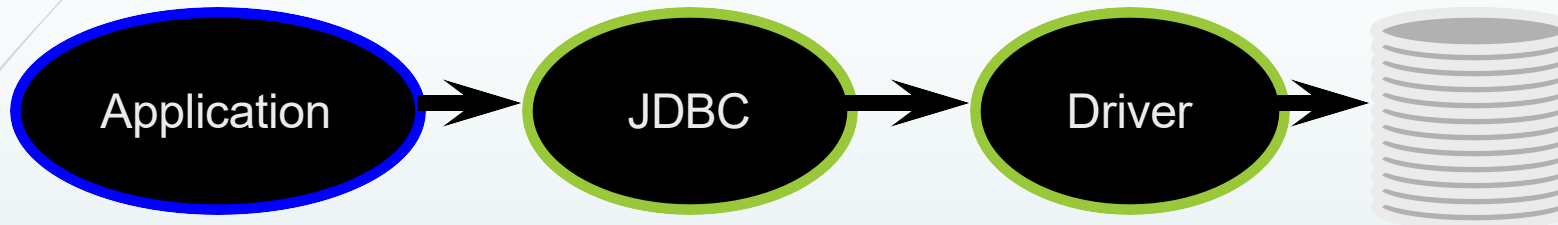
**How java application talk to
databases then ?**

3.5 Overview of the database connectivity

- ▶ JDBC – Java DataBase Connectivity
 - ▶ Industry standard interface for connecting to relational databases from Java
 - ▶ Independent of any DBMS
 - ▶ Allows three things
 - ▶ Establish a connection with Relational databases
 - ▶ Send SQL statements
 - ▶ Process the results
 - ▶ Provides two APIs
 - ▶ API for application writers
 - ▶ API for driver writers

JDBC Architecture

35



- ▶ Java code calls JDBC library
- ▶ JDBC loads a *driver*
- ▶ Driver talks to a particular database
- ▶ Can have more than one driver -> more than one database
- ▶ Ideal: can change database engines without changing any application code

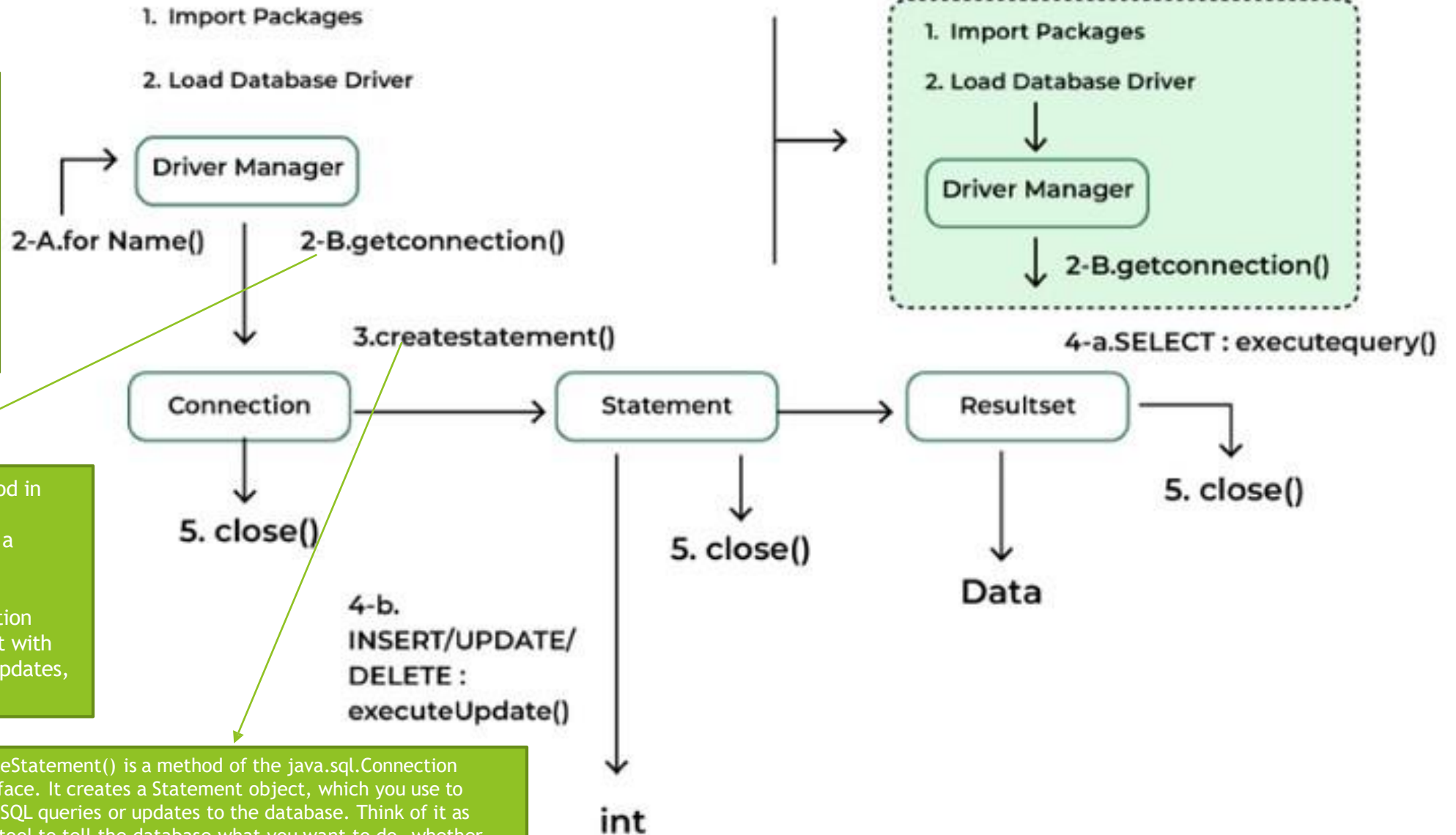
```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
```

Java Database Connectivity

`Class.forName()` is a static method in Java's `java.lang.Class` class. It dynamically loads a class into memory at runtime by its fully qualified name (e.g., `com.mysql.cj.jdbc.Driver`). In the context of JDBC, it was historically used to register a database driver with the `DriverManager` before establishing a connection.

`getConnection()` is a static method in the `java.sql.DriverManager` class. It's your gateway to establishing a connection between your Java application and a database. Once called, it returns a `Connection` object, which you use to interact with the database—running queries, updates, or managing transactions.

`createStatement()` is a method of the `java.sql.Connection` interface. It creates a `Statement` object, which you use to send SQL queries or updates to the database. Think of it as your tool to tell the database what you want to do—whether it's fetching data (`SELECT`), inserting rows (`INSERT`), or updating records (`UPDATE`).



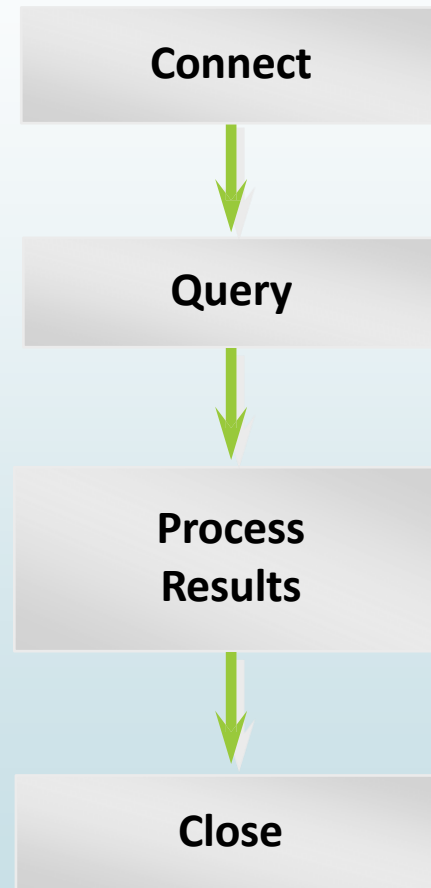
A JDBC Driver

- ▶ Is an interpreter that translates JDBC method calls to vendor-specific database commands

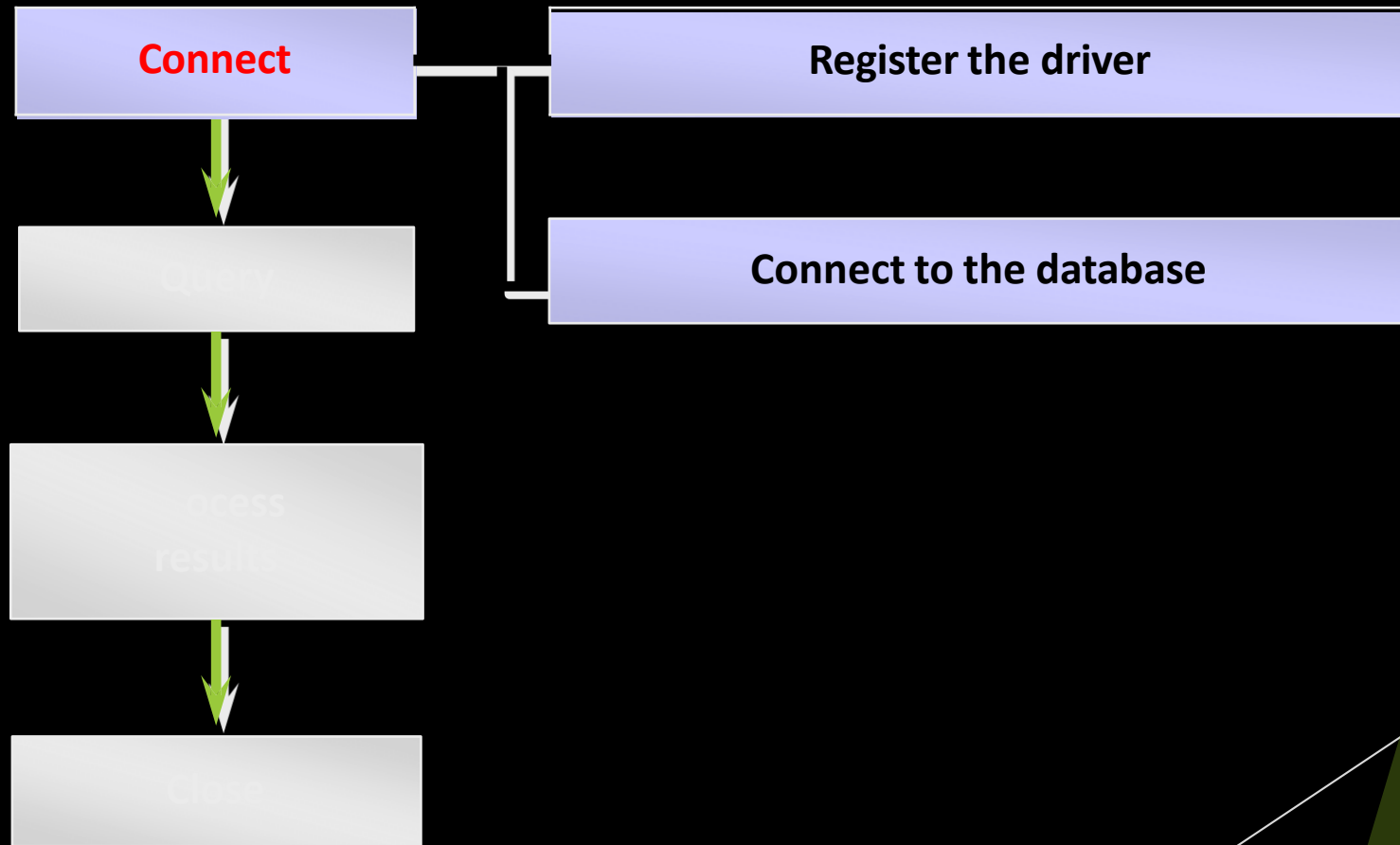


- ▶ Implements interfaces in **java.sql**
- ▶ Can also provide a vendor's extensions to the JDBC standard

Overview of Querying a Database With JDBC



Stage 1: Connect



Registering a Driver

- ▶ In order to begin with, you first need to load the driver or register it before using it in the program. Registration is to be done once in your program.
- ▶ You can register a driver in one of two ways mentioned below as follows:

Class.forName()

Here we load the driver's class file into memory at the runtime.

No need of using new or create objects.

The following example uses `Class.forName()` to load the Oracle driver as shown below as follows:

```
Class.forName("oracle.jdbc.driver.OracleDriver");
```

Statically load driver

```
Class.forName("Driver Name");
```

Use the *jdbc.drivers* system property

Every driver has its own name

For example

mysql - `com.mysql.jdbc.Driver`

Oracle - `oracle.jdbc.driver.OracleDriver`

DriverManager.registerDriver()

`DriverManager` is a Java inbuilt class with a static member `register`.

Here we call the constructor of the driver class at compile time.

The following example uses

`DriverManager.registerDriver()` to register the Oracle driver as shown below:

```
DriverManager.registerDriver(new oracle.jdbc.driver.OracleDriver())
```


Connection to the Database

- ▶ Handled by **DriverManager** class Object

- ▶ Calls `getConnection()` Method

- ▶ Accepts

- ▶ JDBC URL,

- ▶ username

- ▶ password

- ▶ Returns a Connection object

- ▶ Throws **java.sql.SQLException**

After loading the driver, establish connections as shown below as follows:

```
Connection con = DriverManager.getConnection(url,user,password)
```

user: Username from which your SQL command prompt can be accessed.

password: password from which the SQL command prompt can be accessed.

con: It is a reference to the Connection interface.

Url: Uniform Resource Locator which is created as shown below:

JDBC URLs

- ▶ JDBC uses a URL to identify the database connection

jdbc:subprotocol:source

- ▶ Each driver has its own subprotocol
- ▶ Each subprotocol has its own syntax for the source

RDBMS	Database URL format
MySQL	<i><code>jdbc:mysql://hostname:portNumber/databaseName</code></i>
ORACLE	<i><code>jdbc:oracle:thin:@hostname:portNumber:databaseName</code></i>
DB2	<i><code>jdbc:db2:hostname:portNumber/databaseName</code></i>
PostgreSQL	<i><code>jdbc:postgresql://hostname:portNumber/databaseName</code></i>
Java DB/Apache Derby	<i><code>jdbc:derby:databaseName</code> (embedded) <i><code>jdbc:derby://hostname:portNumber/databaseName</code> (network)</i></i>
Microsoft SQL Server	<i><code>jdbc:sqlserver://hostname:portNumber;databaseName=databaseName</code></i>
Sybase	<i><code>jdbc:sybase:Tds:hostname:portNumber/databaseName</code></i>

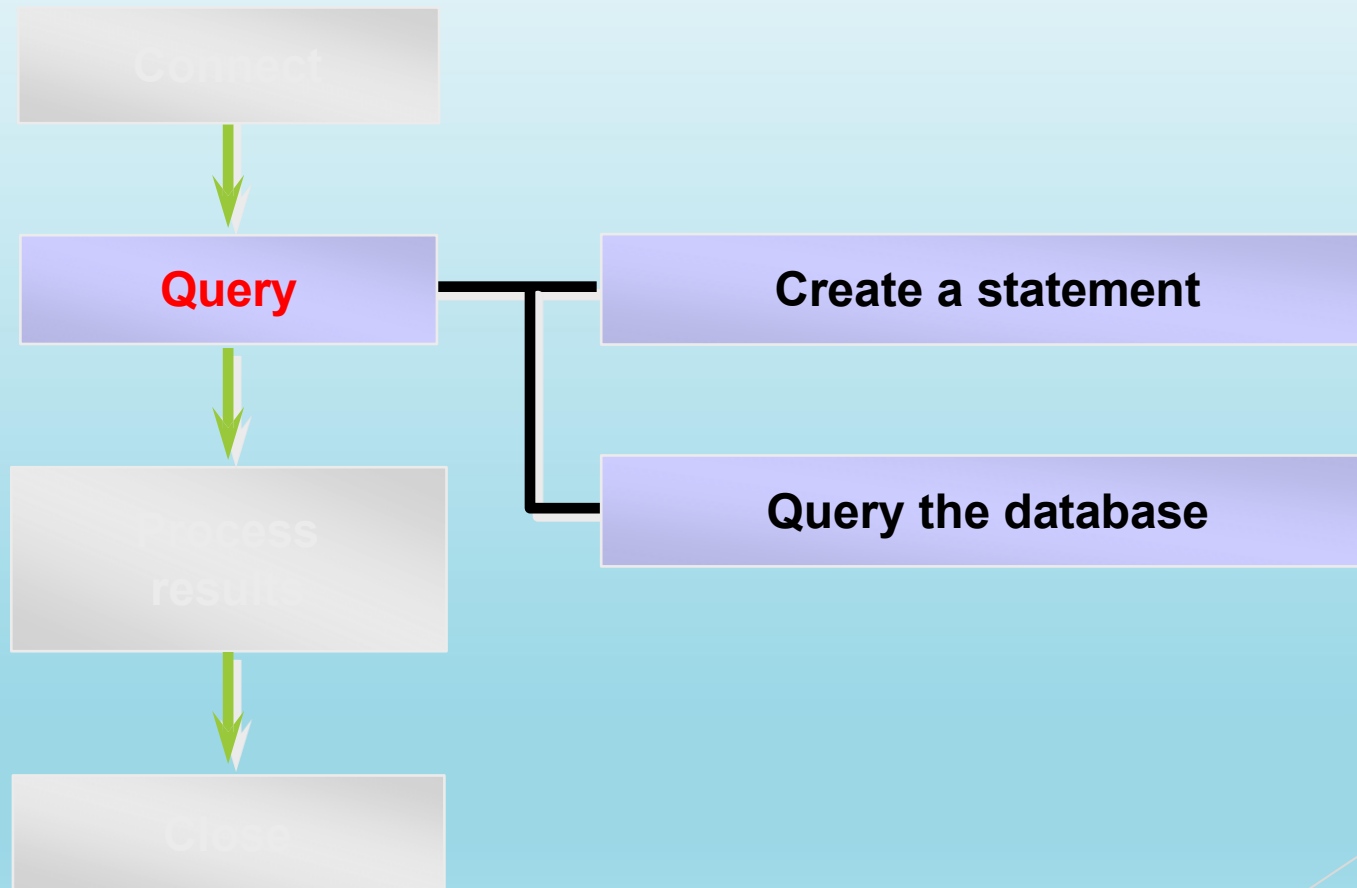
Connection

- ▶ A Connection represents a session with a specific database.
- ▶ Within the context of a Connection, SQL statements are executed and results are returned.
- ▶ Can have multiple connections to a database
- ▶ Also provides “metadata” -- information about the database, tables, and fields
- ▶ Also methods to deal with transactions

Obtaining a Connection

```
String url = jdbc:oracle:thin:@dbserv.cs.siu.edu:1521:cs";  
try {  
    Class.forName ("oracle.jdbc.driver.OracleDriver");  
    Connection con = DriverManager.getConnection(url);  
}  
catch (ClassNotFoundException e)  
    { e.printStackTrace(); }  
catch (SQLException e)  
    { e.printStackTrace(); }
```

Stage 2: Query



Statement

- ▶ A Statement object is used for executing a SQL statement and obtaining the results produced by it.
- ▶ Different types of Statements
 - ▶ Statement
 - ▶ PreparedStatement
 - ▶ CallableStatement

Statement

- ▶ Once a connection is established you can interact with the database.
- ▶ The `PreparedStatement`, `CallableStatement`, and `PreparedStatement` interfaces define the methods that enable you to send SQL commands and receive data from your database.
- ▶ Use of JDBC Statement is as follows:
 - ▶ `Statement st = con.createStatement();`
- ▶ Statement
 - ▶ Used for executing a static SQL statement
 - ▶ Created by `createStatement()` method on the connection object
 - ▶ Returns a new Statement object

PreparedStatement

48

- Creates precompiled SQL statements - more efficient than Statement class
- Can also allow specifying parameters that can be filled during run time
- Usually used if the statements is going to execute more than once
- A PreparedStatement is an interface in the java.sql package that extends Statement.
- It represents a precompiled SQL statement with placeholders (parameters) that you set dynamically.
- Unlike a regular Statement, which executes static SQL strings, PreparedStatement is designed for queries or updates where values change—like user inputs or repeated operations.
- Created by `prepareStatement()` method on the connection object
 - Accepts the query
 - returns a new PreparedStatement object

CallableStatement

- ▶ Many DBMS can store individual or set of SQL statements in a database
 - ▶ Stored Procedures
 - ▶ A stored procedure is a prepared SQL code that you can save, so the code can be reused over and over again.
- ▶ JDBC allows programs to invoke stored procedures using CallableStatement interface
- ▶ A Callable statement object holds parameters for calling stored procedures
- ▶ Created by `prepareCall(String sql)`
 - ▶ Accepts the query
 - ▶ returns a new CallableStatement object

Statement Methods

- ▶ `executeQuery(String)`
 - ▶ Accepts SQL query
 - ▶ Execute a SQL query that returns a single `ResultSet`
- ▶ `executeUpdate(String)`
 - ▶ Execute a SQL `INSERT`, `UPDATE` or `DELETE` statement. Returns the number of rows changed.
- ▶ `Execute()`
 - ▶ Execute a SQL statement that may return multiple results.

```
import java.sql.*;

Class MyClass {
    public static void main(String[] args) throws Exception {
        String url = "jdbc:mysql://localhost:3306/database_name"; // Database details
        String username = "rootgfg"; // MySQL credentials
        String password = "gfg123";
        String query = "select * from students"; // Query to be run

        // Load and register the driver
        Class.forName("com.mysql.cj.jdbc.Driver");

        // Establish connection
        Connection con = DriverManager.getConnection(url, username, password);
        System.out.println("Connection Established successfully");

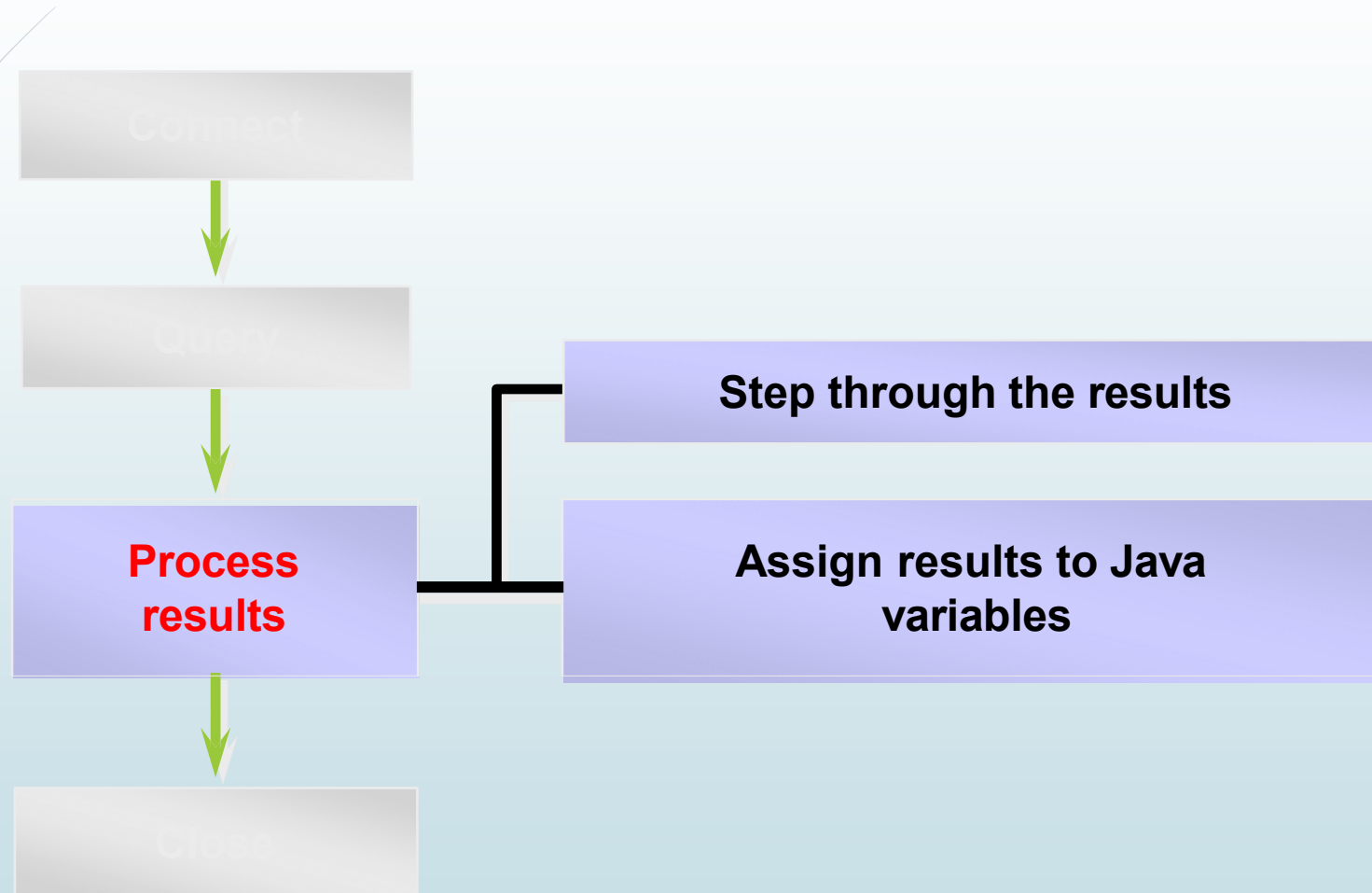
        // Create a statement
        Statement st = con.createStatement();

        // Execute the query
        ResultSet rs = st.executeQuery(query);

        // Process the results
        while (rs.next()) {
            String name = rs.getString("name"); // Retrieve name from db
            System.out.println(name); // Print result on console
        }

        // Close the statement and connection
        st.close();
        con.close();
        System.out.println("Connection Closed....");
    }
}
```

Stage 3: Process the Results



ResultSet

- ▶ A ResultSet provides access to a table of data generated by executing a Statement.
- ▶ Only one ResultSet per Statement can be open at once.
- ▶ The table rows are retrieved in sequence.
- ▶ A ResultSet maintains a cursor pointing to its current row of data.
- ▶ The 'next' method moves the cursor to the next row.

A ResultSet is an interface in the java.sql package that represents the data returned from a database query, typically a SELECT statement.

It acts like a table of rows and columns, letting you iterate over the results and extract values (e.g., strings, integers) from each row.

Think of it as your bridge to the data fetched from the database.

ResultSet Methods

- ▶ boolean **next()**
 - ▶ activates the next row
 - ▶ the first call to next() activates the first row
 - ▶ returns false if there are no more rows

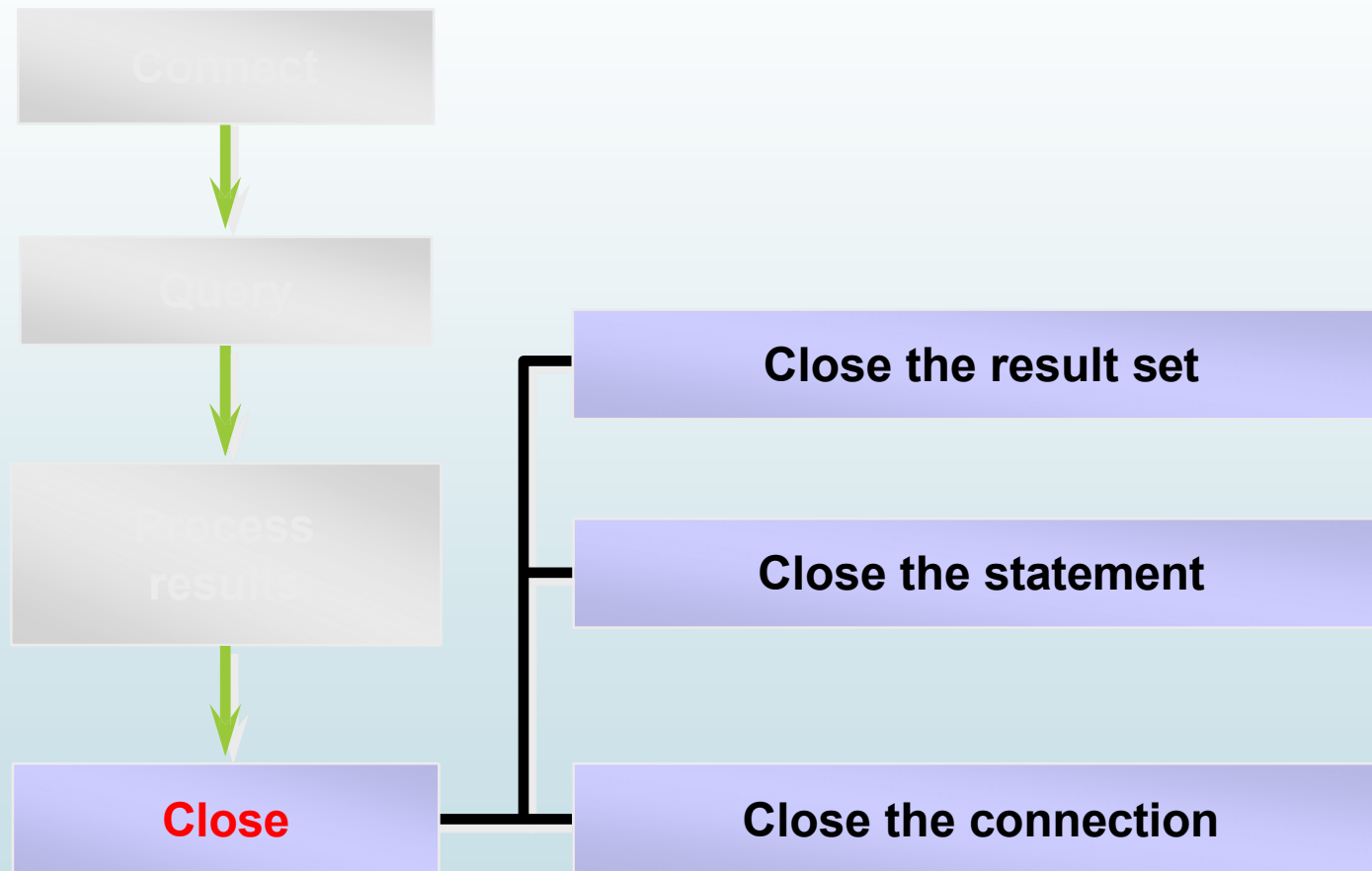
ResultSet Methods

- ▶ **Type `getType(int columnIndex)`**
 - ▶ returns the given field as the given type
 - ▶ fields indexed starting at 1 (not 0)
- ▶ **Type `getType(String columnName)`**
 - ▶ same, but uses name of field
 - ▶ less efficient
- ▶ **`int findColumn(String columnName)`**
 - ▶ looks up column index given column name

ResultSet Methods

- `String getString(String columnName)`
- `boolean getBoolean(String columnName)`
- `byte getByte(String columnName)`
- `short getShort(String columnName)`
- `int getInt(String columnName)`
- `long getLong(String columnName)`
- `float getFloat(String columnName)`
- `double getDouble(String columnName)`
- `Date getDate(String columnName)`
- `Time getTime(String columnName)`
- `Timestamp getTimestamp(String columnName)`

Stage 4: Close



Close

- Close the ResultSet object
 - `void close()`
 - disposes of the ResultSet
 - allows you to re-use the Statement that created it
 - automatically called by most Statement methods
- ▶ Close the Statement Object
 - `void close()`
- ▶ Close the connection
 - ▶ `void close()`

- ▶ Step 1: import java.sql Package
- ▶ Step2: Load and register the driver
- ▶ Step 3:Connect to the database
- ▶ Step 4: Create statements
- ▶ Step 5: Execute statements & Process the Result
- ▶ Step 6: close ResultSet and Statement
- ▶ Step 7: Close the connection

JDBC Object Classes

- **DriverManager**
 - Loads, chooses drivers
 - **Driver**
 - connects to actual database
 - **Connection**
 - a series of SQL statements to and from the DB
 - **Statement**
 - a single SQL statement
 - **ResultSet**
 - the records returned from a Statement
- The following program demonstrates how to connect to a MySQL database using JDBC, execute a query, retrieve the results, and display them in a Java program.

Overview of what the code does

1. The program loads the MySQL JDBC driver using the **Class.forName()** method.
2. It then connects to the MySQL database using the **DriverManager.getConnection()** method and a JDBC URL that specifies the database name, username, and password.
3. It creates a **Statement** object to execute SQL queries.
4. It executes a **SELECT** query on the "Authors" table using the **executeQuery()** method and stores the results in a **ResultSet** object.
5. It iterates through the **ResultSet** using the **next()** method and retrieves the values of the "authorId", "firstName", and "lastName" columns using the **getInt()** and **getString()** methods.
6. It displays the retrieved data using the **System.out.println()** method.
7. It closes the **ResultSet**, **Statement**, and **Connection** objects using the **close()** method.

Example

```
import java.sql.*;

public class DisplayAuthors {
    public static void main(String[] args) {
        try {
            // Load the JDBC driver
            Class.forName("com.mysql.cj.jdbc.Driver");
            // Connect to the database
            String url =
                "jdbc:mysql://localhost:3306/Books?user=myuser&password=mypassword";
            Connection conn = DriverManager.getConnection(url);
            // Create a Statement object to execute SQL queries
            Statement stmt = conn.createStatement();

            // Execute the query and store the results in a ResultSet object
            ResultSet rs = stmt.executeQuery("SELECT authorId, firstName,
            lastName FROM Authors");
```

Example

63

```
while (rs.next()) {  
    int authorId = rs.getInt("authorId");  
    String firstName = rs.getString("firstName");  
    String lastName = rs.getString("lastName");  
    System.out.println("Author ID: " + authorId + ", First Name: " +  
        firstName + ", Last Name: " + lastName); }  
// Close the ResultSet, Statement, and Connection objects  
    rs.close();  
    stmt.close();  
    conn.close();  
  
} catch (Exception e) {  
    e.printStackTrace();  
} } }
```

Overview of what the code does

1. The program loads the MySQL JDBC driver using the **Class.forName()** method.
2. It then connects to the MySQL database using the **DriverManager.getConnection()** method and a JDBC URL that specifies the database name, username, and password.
3. It creates a **PreparedStatement** object to execute SQL statements. In this case, the SQL statement is an **INSERT** statement that inserts a new record into the "Authors" table with a first name of "John" and a last name of "Doe". The ? placeholders are used to specify parameter values later.
4. It sets the parameter values for the **INSERT** statement using the **setString()** method.
5. It executes the **INSERT** statement using the **executeUpdate()** method and gets the number of rows affected.
6. It displays a message indicating how many rows were inserted.
7. It closes the **PreparedStatement** and **Connection** objects using the **close()** method.

Insert a single record

```
import java.sql.*;

public class InsertAuthor {
    public static void main(String[] args) {
        try { // Load the JDBC driver
            Class.forName("com.mysql.cj.jdbc.Driver");
            // Connect to the database String url =
            "jdbc:mysql://localhost:3306/Books?user=myuser&password=
            mypassword"; Connection conn =
            DriverManager.getConnection(url); // Create a
            PreparedStatement object to execute SQL statements
            String query = "INSERT INTO Authors (firstName, lastName)
            VALUES (?, ?)";

            PreparedStatement pstmt = conn.prepareStatement(query);
```

Insert a single record

```
// Set the parameter values for the query
pstmt.setString(1, "John");
pstmt.setString(2, "Doe");
// Execute the query and get the number of rows affected
int rows = pstmt.executeUpdate();
System.out.println(rows + " row(s) inserted.");
// Close the PreparedStatement and Connection objects pstmt.close();
conn.close();
} catch (Exception e) {
e.printStackTrace();
} } }
```

Overview of what the update code does

1. The program loads the MySQL JDBC driver using the **Class.forName()** method.
2. It then connects to the MySQL using **DriverManager.getConnection()** method and a JDBC URL that specifies the database name, username, and password.
3. It creates a **PreparedStatement** object to execute SQL statements. In this case, the SQL statement is an **UPDATE** statement that updates the "firstName" and "lastName" columns of the record with an "authorId" of 1 to "Jane" and "Doe", respectively. The ? placeholders are used to specify parameter values later.
4. It sets the parameter values for the **UPDATE** statement using the **setString()** and **setInt()** methods.
5. It executes the **UPDATE** statement using the **executeUpdate()** method and gets the number of rows affected.
6. It displays a message indicating how many rows were updated.
7. It closes the **PreparedStatement** and **Connection** objects using the **close()** method.

Update a single record

```
import java.sql.*;

public class UpdateAuthor {

    public static void main(String[] args) {
        try { // Load the JDBC driver
            Class.forName("com.mysql.cj.jdbc.Driver");

            // Connect to the database String url =
            "jdbc:mysql://localhost:3306/Books?user=myuser&password=mypas
            sword";

            Connection conn = DriverManager.getConnection(url);

            // Create a PreparedStatement object to execute SQL statements
            String query = "UPDATE Authors SET firstName = ?, lastName = ?
            WHERE authorId = ?";

            PreparedStatement pstmt = conn.prepareStatement(query);
```

Update a single record

```
// Set the parameter values for the query
pstmt.setString(1, "Jane");
pstmt.setString(2, "Doe");
pstmt.setInt(3, 1);
// Execute the query and get the number of rows affected
int rows = pstmt.executeUpdate();
System.out.println(rows + " row(s) updated.");
// Close the PreparedStatement and Connection objects pstmt.close();
conn.close();
} catch (Exception e) {
e.printStackTrace();
} } }
```



Question ? ? ?